



Prediksi Ketepatan Waktu Pengiriman Berdasarkan Data Transaksi dan Produk

Proyek ini bertujuan memprediksi apakah suatu pengiriman akan sampai tepat waktu atau terlambat menggunakan data transaksi pelanggan dan logistik. Dataset berasal dari kompetisi berbasis file train.csv dan test.csv.



Tujuan & Ruang Lingkup

Tujuan Prediksi Format Data & Target

Proyek ini bertujuan memprediksi apakah suatu pengiriman akan sampai tepat waktu atau terlambat menggunakan data transaksi pelanggan dan logistik. Dataset berasal dari kompetisi berbasis file train.csv dan test.csv.



Preview Data set

-- Informasi Kolom dan Tipe Data ---

	Kolom	Tipe Data
0	id	int64
1	blok_gudang	object
2	mode_pengiriman	object
3	panggilan_layanan_pelanggan	int64
4	rating_pelanggan	int64
5	harga_produk	float64
6	pembelian_sebelumnya	int64
7	kepentingan_produk	object
8	jenis_kelamin	object
9	diskon_ditawarkan	int64
10	berat_dalam_gram	float64
11	pengiriman_tepat_waktu	int64

- id - Identifikasi unik setiap entri pesanan.
- blok_gudang - Blok lokasi gudang tempat produk disimpan.
- mode_pengiriman - Metode pengiriman yang digunakan.
- panggilan_layanan_pelanggan - Jumlah interaksi pelanggan dengan layanan pelanggan.
- rating_pelanggan - Rating atau penilaian yang diberikan oleh pelanggan terhadap layanan (1-5).
- harga_produk - Harga dari produk yang dibeli.
- pembelian_sebelumnya - Jumlah transaksi sebelumnya oleh pelanggan tersebut.
- kepentingan_produk - Tingkat kepentingan produk bagi pelanggan (misalnya: Rendah, Sedang, Tinggi).
- jenis_kelamin - Jenis kelamin pelanggan (Laki-laki atau Perempuan).
- diskon_ditawarkan - Persentase diskon yang diberikan pada produk.
- berat_dalam_gram - Berat produk dalam satuan gram.
- pengiriman_tepat_waktu - Kolom target (0: Terlambat, 1: Tepat waktu).

Metodologi yang digunakan

- 1 **EDA & Data Preprocessing**
- 2 **Feature Selection**
- 3 **Modelling**
- 4 **Evaluation**

Exploratory Data Analysis & Data Preprocessing



Encoding Fitur Kategorikal

Alasan Kenapa Encoding Terlebih Dahulu

Encoding bertujuan untuk mengubah data kategorikal (seperti jenis kelamin, metode pengiriman, dll) menjadi format numerik agar bisa dianalisis dan digunakan oleh model machine learning. Alasan kenapa encoding di lakukan di awal adalah:

- Kita ingin membandingkan korelasi antar fitur dalam dataset. Matriks korelasi hanya bisa dihitung untuk data numerik, sehingga fitur kategorikal perlu diubah terlebih dahulu ke bentuk numerik.
- Dengan melakukan encoding, kita dapat lebih mudah melihat hubungan antar fitur dan target dalam matriks korelasi.
- Proses ini juga membantu kita menghapus fitur yang tidak berkorelasi dengan target atau fitur lainnya yang tidak memberikan kontribusi dalam prediksi.

Kolom Kategorikal

Terdapat 4 kolom kategorikal dalam dataset ini yang perlu di-encode, yaitu:

- blok_gudang
- mode_pengiriman
- kepentingan_produk
- jenis_kelamin
-

Mengapa Perlu Encoding?

- Model machine learning tidak dapat menangani data kategorikal secara langsung, karena model hanya dapat bekerja dengan data numerik.

- Informasi Kolom dan Tipe Data ---

	Kolom	Tipe Data
0	id	int64
1	blok_gudang	object
2	mode_pengiriman	object
3	panggilan_layanan_pelanggan	int64
4	rating_pelanggan	int64
5	harga_produk	float64
6	pembelian_sebelumnya	int64
7	kepentingan_produk	object
8	jenis_kelamin	object
9	diskon_ditawarkan	int64
10	berat_dalam_gram	float64
11	pengiriman_tepat_waktu	int64

- Statistik Deskriptif ---

	id	panggilan_layanan_pelanggan
--	----	-----------------------------

Teknik Yang Digunakan

- Informasi Kolom dan Tipe Data ---

	Kolom	Tipe Data
0	id	int64
1	blok_gudang	object
2	mode_pengiriman	object
3	panggilan_layanan_pelanggan	int64
4	rating_pelanggan	int64
5	harga_produk	float64
6	pembelian_sebelumnya	int64
7	kepentingan_produk	object
8	jenis_kelamin	object
9	diskon_ditawarkan	int64
10	berat_dalam_gram	float64
11	pengiriman_tepat_waktu	int64

- Statistik Deskriptif ---

	id	panggilan_layanan_pelanggan
--	----	-----------------------------

Kolom

jenis_kelamin, blok_gudang, mode_pengiriman, kepentingan_produk

```
from sklearn.preprocessing import LabelEncoder, StandardScaler
le = LabelEncoder()

# Fit and transform the training data
le.fit(df_train['jenis_kelamin'])
df_train['jenis_kelamin'] = le.transform(df_train['jenis_kelamin'])

# Encode the test data
df_test['jenis_kelamin'] = le.transform(df_test['jenis_kelamin'])

# Print the first few rows of the training and testing data
print(df_train.head())
print(df_test.head())
```

Label

- Jumlah kategori sedikit di masing-masing kolom
 - Tidak ada urutan antar kategori (non-ordinal) pada kolom seperti jenis_kelamin (Laki-laki/Perempuan) dan blok_gudang
 - Untuk mode_pengiriman, meskipun memiliki urutan, kita tetap menggunakan Label Encoding karena untuk kemudahan dan konsistensi dalam pengolahan data.
- Penggunaan Label Encoding pada kolom yang memiliki urutan juga umum dilakukan di banyak kasus praktis.

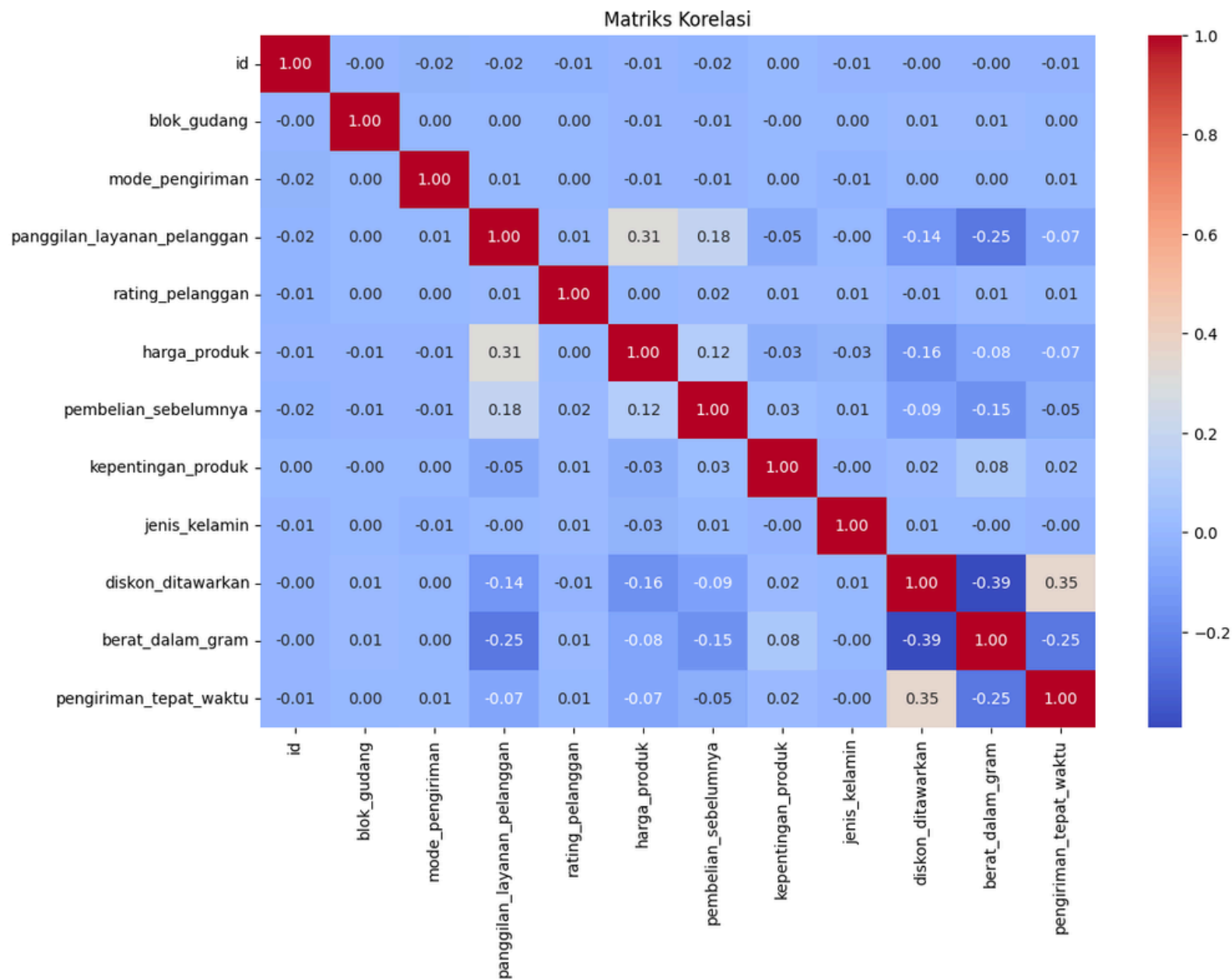


Sinkronisa

si
Encoding dilakukan pada train dan test set secara konsisten untuk memastikan data yang digunakan dalam model adalah seragam.

Pearson Correlation

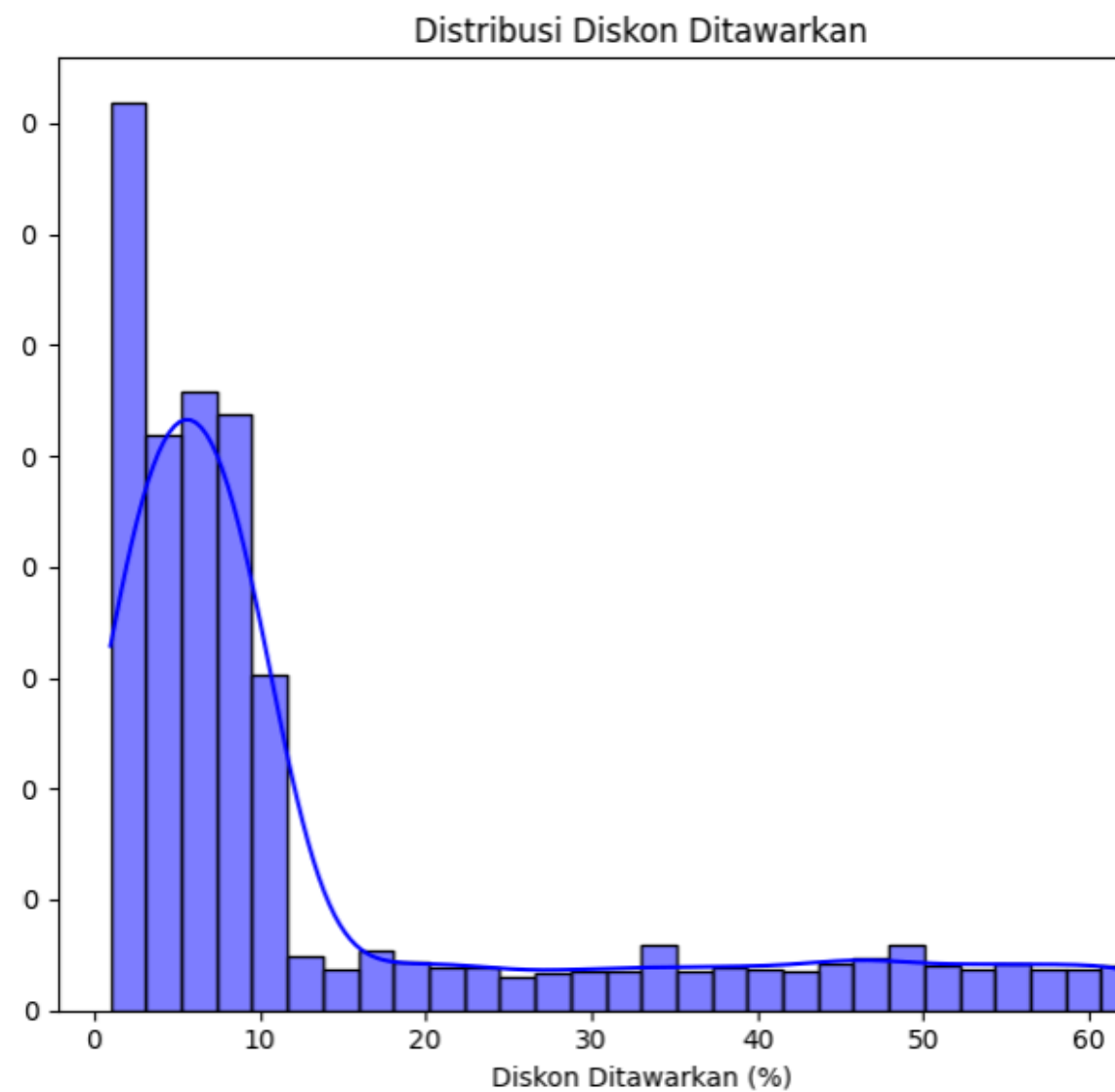
Insight Utama dari Matriks Korelasi



- Prediktor Paling Relevan:
 - diskon_ditawarkan vs pengiriman_tepat_waktu = 0.35 (positif sedang)
 - berat_dalam_gram vs pengiriman_tepat_waktu = -0.25 (negatif sedang)
- Prediktor Lemah Lainnya
 - blok_gudang vs pengiriman_tepat_waktu = -0.07
 - rating_pelanggan vs pengiriman_tepat_waktu = 0.008
- Prediktor Tanpa Korelasi:
 - id vs pengiriman_tepat_waktu = 0.00
 - blok_gudang vs pengiriman_tepat_waktu = 0.00
 - jenis_kelamin vs pengiriman_tepat_waktu = 0.00

Eksplorasi Data & Analisis Awal

Distribusi Data Variabel Prediktor



Distribusi Diskon Ditawarkan:

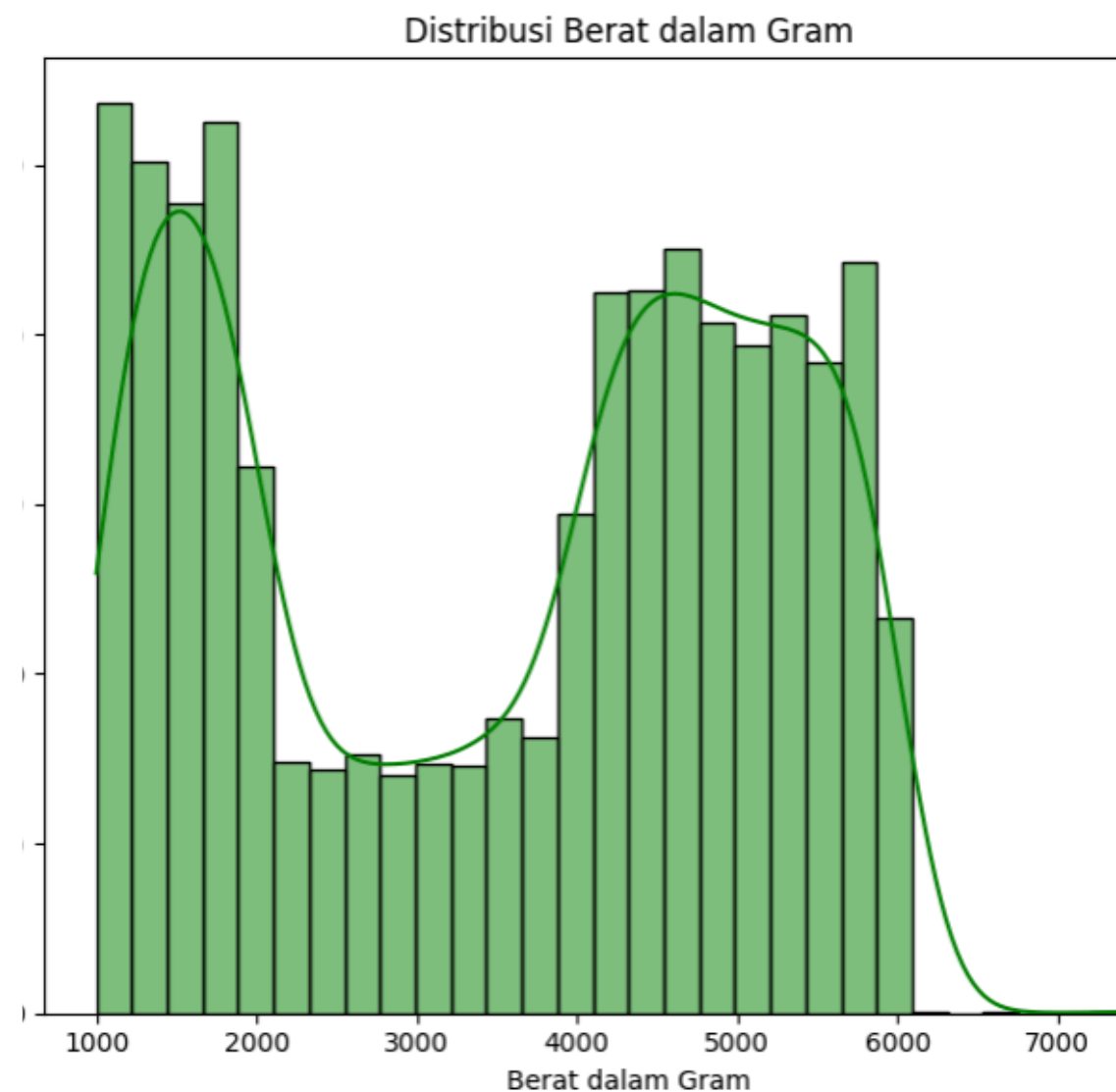
Distribusi ini tampaknya mengikuti distribusi miring ke kanan (right-skewed) dengan puncak yang sangat jelas di sekitar nilai rendah (0–10%).

Karakteristik yang Terlihat:

- Puncak di sekitar nilai 0 – Menunjukkan bahwa nilai 0 atau diskon yang sangat kecil adalah yang paling sering terjadi dalam dataset ini.
- Ekor yang memanjang ke kanan – Terdapat beberapa nilai yang lebih besar, namun jumlahnya jauh lebih sedikit dibandingkan nilai kecil.
- Distribusi sangat miring ke kanan (positively skewed) – Ini berarti mayoritas data berada di sisi kiri, namun ada beberapa observasi dengan nilai yang jauh lebih tinggi.

Eksplorasi Data & Analisis Awal

Distribusi Data Variabel Prediktor



Distribusi Berat dalam Gram

Distribusi ini tampak tidak simetris dan miring ke kanan (right-skewed).

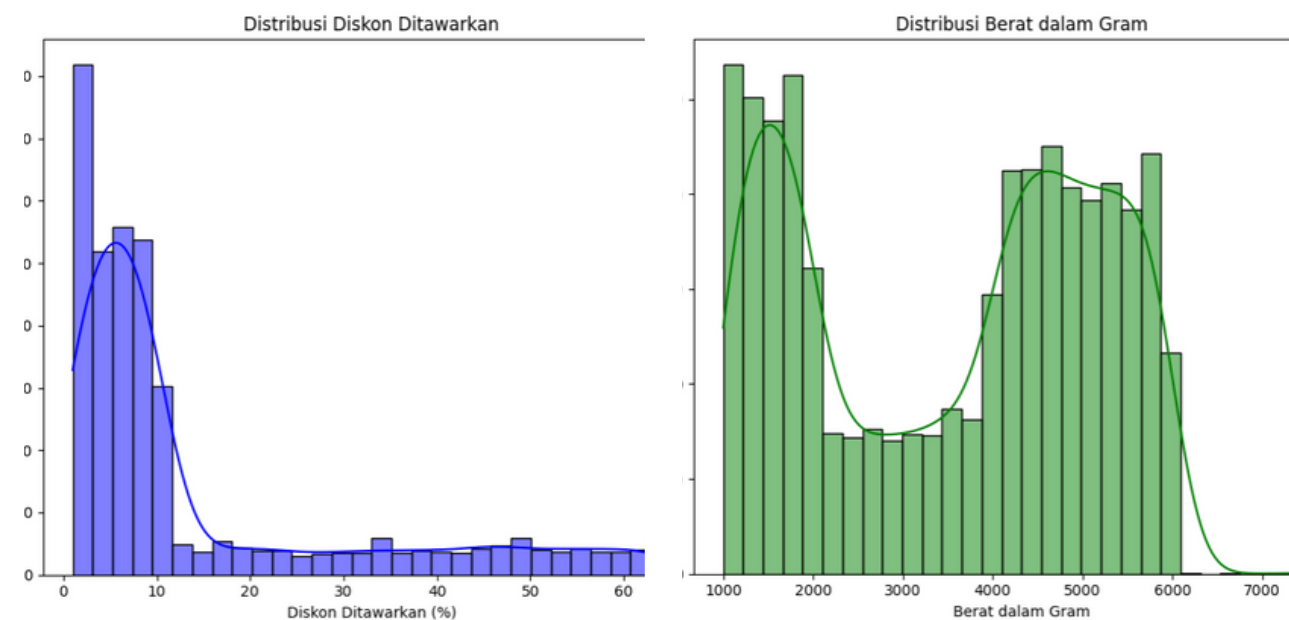
Karakteristik yang Terlihat:

- Puncak di sekitar 3000-4000 gram – menunjukkan bahwa sebagian besar data memiliki berat produk di sekitar nilai ini.
- Ekor yang memanjang ke kanan – ada beberapa nilai yang memiliki berat sangat tinggi (di atas 5000 gram).
- Distribusi tidak simetris – menunjukkan adanya skewness ke kanan, artinya ada lebih banyak produk dengan berat rendah dan sedikit produk dengan berat sangat tinggi.



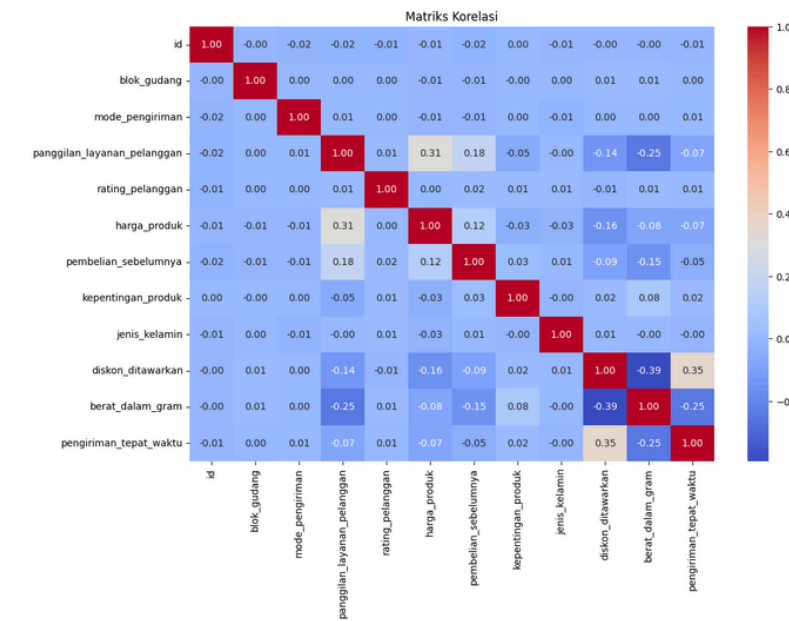
Kesimpulan

Distribusi Data



Distribusi data menunjukkan pola yang skewed dengan kecenderungan right-skewed pada beberapa variabel, terutama pada kolom `diskon_ditawarkan` yang banyak memiliki nilai 0, yang mungkin disebabkan oleh penggantian missing value dengan angka 0. Kolom `berat_dalam_gram` menunjukkan distribusi dengan puncak di sekitar 3000-4000 gram dan sedikit skewness ke kanan, dengan beberapa produk berat sangat tinggi. Ekor panjang ke kanan menunjukkan bahwa ada beberapa produk yang sangat berat atau dengan diskon tinggi yang dapat mempengaruhi pengiriman tepat waktu.

Korelasi Fitur terhadap Target



Berdasarkan analisis korelasi, langkah selanjutnya adalah melakukan feature engineering dengan fokus pada kolom `diskon_ditawarkan` dan `berat_dalam_gram`, untuk menghasilkan fitur baru yang lebih relevan. Fitur yang tidak berkorelasi, seperti `blok_gudang`, `rating_pelanggan`, dan `jenis_kelamin`, akan dihapus, kecuali `id` yang tetap dipertahankan untuk keperluan submission di Kaggle.

Handling Missing Value

```
--- Missing Values in df_train ---
id                                0.000000
blok_gudang                       0.000000
mode_pengiriman                   0.000000
panggilan_layanan_pelanggan       0.000000
rating_pelanggan                   0.000000
harga_produk                      0.887300
pembelian_sebelumnya              0.000000
kepentingan_produk                1.403788
jenis_kelamin                     0.000000
diskon_ditawarkan                 0.000000
berat_dalam_gram                  1.364058
pengiriman_tepat_waktu            0.000000
dtype: float64
```

Handle Missing Values pada data Train

Rata-rata proporsi missing value: 1.5%, dan semuanya berada di bawah threshold 5%

✦ Keputusan:

- Kita tidak menghapus baris, tetapi melakukan imputasi terhadap nilai yang hilang.

📝 Alasan:

- Proporsi missing value yang rendah (<5%) membuat penghapusan data menjadi kurang efisien
- Imputasi menjaga jumlah data dan mencegah hilangnya informasi

Karena kolom yang mengalami missing values di train dan test adalah sama,

Kita menggunakan metode imputasi yang sama untuk kedua dataset (train & test) agar proses preprocessing konsisten dan menghindari data leakage..

Metode Imputasi

```
.impute import SimpleImputer

# kategori (kepentingan_produk) - imputasi
cat_imputer = SimpleImputer(strategy='most_frequent')
df_train['kepentingan_produk'] = cat_imputer.fit_transform(df_train['kepentingan_produk'])
df_test['kepentingan_produk'] = cat_imputer.transform(df_test['kepentingan_produk'])

# numerik (berat_dalam_gram, harga_produk)
num_imputer = SimpleImputer(strategy='median')
df_train[['berat_dalam_gram', 'harga_produk']] = num_imputer.fit_transform(df_train[['berat_dalam_gram', 'harga_produk']])
df_test[['berat_dalam_gram', 'harga_produk']] = num_imputer.transform(df_test[['berat_dalam_gram', 'harga_produk']])

# tidak ada missing values lagi
print("Missing values in train set:", df_train.isnull().sum())
print("Missing values in test set:", df_test.isnull().sum())
```

kepentingan_produk (kategori): diisi dengan mode karena menunjukkan preferensi paling umum

```
.impute import SimpleImputer

# kategori (kepentingan_produk) - imputasi
cat_imputer = SimpleImputer(strategy='most_frequent')
df_train['kepentingan_produk'] = cat_imputer.fit_transform(df_train['kepentingan_produk'])
df_test['kepentingan_produk'] = cat_imputer.transform(df_test['kepentingan_produk'])

# numerik (berat_dalam_gram, harga_produk)
num_imputer = SimpleImputer(strategy='median')
df_train[['berat_dalam_gram', 'harga_produk']] = num_imputer.fit_transform(df_train[['berat_dalam_gram', 'harga_produk']])
df_test[['berat_dalam_gram', 'harga_produk']] = num_imputer.transform(df_test[['berat_dalam_gram', 'harga_produk']])

# tidak ada missing values lagi
print("Missing values in train set:", df_train.isnull().sum())
print("Missing values in test set:", df_test.isnull().sum())
```

berat_dalam_gram, harga_produk (numerik): diisi dengan median karena lebih tahan terhadap outlier



Alasan tidak menggunakan drop: data hilang hanya $\pm 4\%$, dan imputasi menjaga jumlah data tetap maksimal



Menghapus Kolom Tidak Ada Korelasi

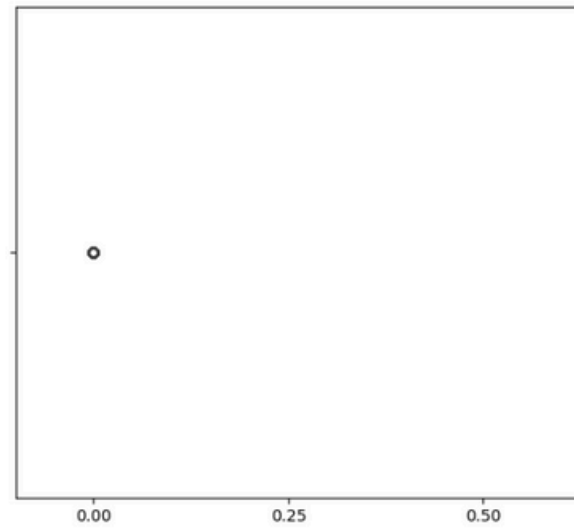
```
kolom yang akan dihapus  
dan 'id' ke dalam daftar kolom yang akan dihapus  
drop = ['jenis_kelamin', 'blok_gudang']  
kolom dari df_train  
df_train.drop(columns=columns_to_drop)  
kolom dari df_test  
df_test.drop(columns=columns_to_drop)  
- df_train setelah menghapus kolom --  
df_train.head()  
- df_test setelah menghapus kolom --  
df_test.head()  
- df_test setelah menghapus kolom --
```

Kolom jenis_kelamin dan blok_gudang dihapus karena kontribusinya terhadap prediksi tidak ada; keduanya menunjukkan korelasi nol terhadap target. Selain itu, distribusi nilainya bersifat acak dan tidak menunjukkan pola yang konsisten terhadap keterlambatan pengiriman. Menghapus fitur dengan relevansi rendah membantu menyederhanakan model dan mengurangi risiko noise.

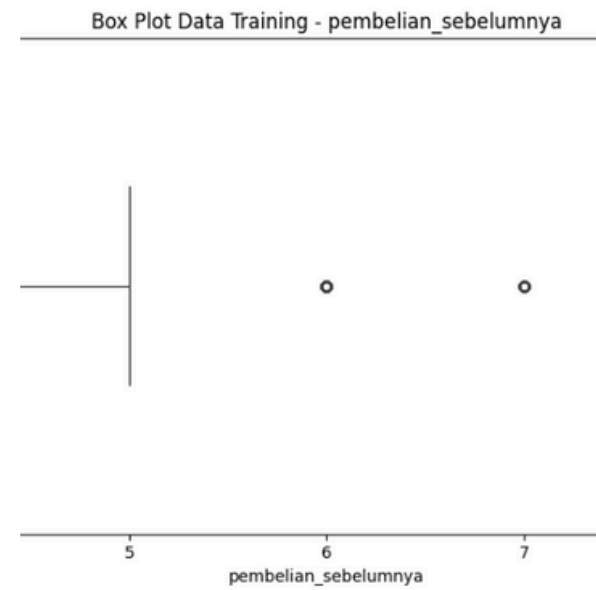


Deteksi & Penanganan Outlier

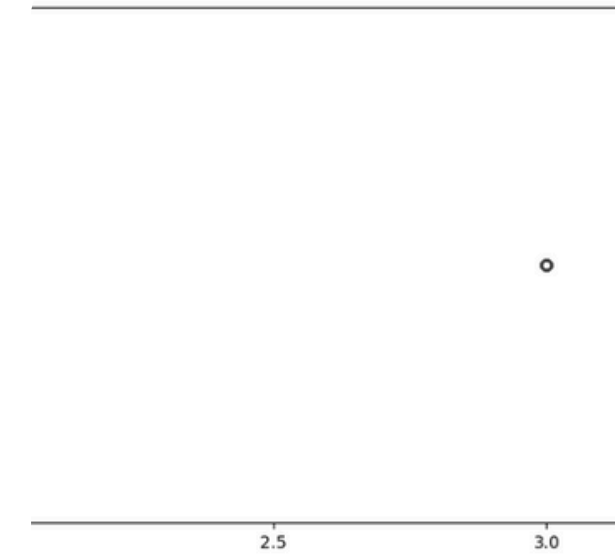
Kolom Outlier



Mode Pengiriman



Pembelian Sebelumnya



Kepentingan Produk



Deteksi & Penanganan Outlier

Kolom Outlier



Diskon Ditawarkan

Teknik Yang Digunakan

```
outlier handling untuk df_train
-- Outlier Handling pada df_train --
outlier_cols:
in ['mode_pengiriman', 'pembelian']
Transformasi logaritmik (tambahkan log1p)
df_train[col] = np.log1p(df_train[col])
print(f" - Kolom '{col}': Transformasi logaritmik")
col == 'diskon_ditawarkan':
print(f" - Kolom '{col}': Capping")

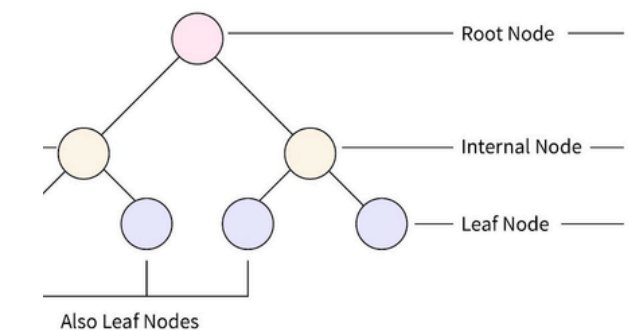
Lakukan capping setelah transformasi
df_train = cap_outliers_iqr(df_train, col)
```

Beberapa fitur seperti pembelian_sebelumnya, mode_pengiriman, dan kepentingan_produk menunjukkan distribusi yang skewed dan mengandung outlier signifikan. Oleh karena itu, fitur-fitur ini ditransformasi menggunakan logaritma natural ($\log_1 p$) untuk menekan dampak nilai ekstrem dan menormalkan distribusinya.

```
outlier handling untuk df_test
--- Outlier Handling pada df_test ---
outlier_cols:
in ['mode_pengiriman', 'pembelian']
Transformasi logaritmik (tambahkan log1p)
df_test[col] = np.log1p(df_test[col])
print(f" - Kolom '{col}': Transformasi logaritmik")
col == 'diskon_ditawarkan':
print(f" - Kolom '{col}': Capping")

Lakukan capping setelah transformasi
df_test = cap_outliers_iqr(df_test, col)
```

Setelah transformasi log, serta untuk fitur lain seperti diskon_ditawarkan, diterapkan teknik capping berbasis IQR. Capping dilakukan dengan membatasi nilai di luar rentang $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$ agar outlier tidak terlalu memengaruhi proses pelatihan model, tanpa harus menghapus baris data.



Pendekatan ini diterapkan secara konsisten pada data training dan testing. Kombinasi log transform dan capping dianggap cukup efektif, terutama karena model yang digunakan (Gradient Boosting dan XGBoost) cukup robust terhadap outlier namun tetap diuntungkan dari distribusi yang lebih stabil.



Scaling

_train setelah Scaling ---

mode_pengiriman	panggilan_layanan_pelanggan	rating_pelanggan
0.693147	-0.907916	1.423633
0.693147	-0.907916	0.714072
0.693147	1.732266	-0.705051
0.693147	2.612326	1.423633
0.693147	-1.787976	1.423633

harga_produk	pembelian_sebelumnya	kepentingan_produk	diskon_ditawarkan
-0.611984	1.609438	0.693147	1.9
0.970547	1.098612	0.693147	1.9
1.740989	1.791759	0.693147	-0.7
0.699850	1.791759	0.693147	1.3
-1.465718	1.386294	1.386294	1.2

berat_dalam_gram	pengiriman_tepat_waktu
-1.281930	1
-0.809271	1
-1.405339	0
-1.517025	1
-1.434958	1

Scaling pada Data Numerik

Karena model machine learning seperti logistic regression, SVM, atau neural networks sensitif terhadap skala fitur, maka diperlukan proses scaling untuk memastikan semua fitur berada pada skala yang sebanding.

📌 Keputusan:

- Kita menggunakan metode StandardScaler untuk menstandarkan fitur numerik pada data train dan test.

📝 Alasan:

- StandardScaler mengubah distribusi data menjadi mean = 0 dan standar deviasi = 1
- Menjaga bentuk distribusi asli fitur (tidak memaksa ke rentang tertentu seperti MinMaxScaler)

Kenapa Perlu Scaling?

Membantu model bekerja lebih stabil, cepat konvergen, dan menghasilkan bobot fitur yang lebih seimbang

Feature Selection



Feature Engineering

```
df_test['produk_ringan'] = (df_test['berat_dalam_gram'] < 1000).astype(int)

# 4. diskon_tinggi
df_train['diskon_tinggi'] = (df_train['diskon_ditawarkan'] > 50).astype(int)
df_test['diskon_tinggi'] = (df_test['diskon_ditawarkan'] > 50).astype(int)

# 5. skor_pelanggan
df_train['skor_pelanggan'] = df_train['rating_pelanggan'] - df_train['panggi']
df_test['skor_pelanggan'] = df_test['rating_pelanggan'] - df_test['panggi']

# 6. high_value_order
df_train['high_value_order'] = ((df_train['harga_produk'] > 10000) & (df_train['jumlah_order'] > 5)).astype(int)
df_test['high_value_order'] = ((df_test['harga_produk'] > 10000) & (df_test['jumlah_order'] > 5)).astype(int)

# 7. is_frequent_buyer
df_train['is_frequent_buyer'] = (df_train['pembelian_sebelumnya'] > 5).astype(int)
df_test['is_frequent_buyer'] = (df_test['pembelian_sebelumnya'] > 5).astype(int)

# 8. produk_penting_dan_ringan
df_train['produk_penting_dan_ringan'] = ((df_train['kepentingan_produk'] > 3) & (df_train['berat_dalam_gram'] < 1000)).astype(int)
df_test['produk_penting_dan_ringan'] = ((df_test['kepentingan_produk'] > 3) & (df_test['berat_dalam_gram'] < 1000)).astype(int)
```



Feature Scaling pada Data Numerik

Tujuan dari feature engineering sendiri adalah menambahkan fitur baru yang relevan agar model dapat mengenali pola kompleks yang tidak ditangkap oleh fitur asli.

✦ Keputusan:

- Kita membuat beberapa fitur turunan berbasis kombinasi, logika kondisi, dan rasio antar kolom dari dataset awal.

📝 Alasan:

- Model tree-based seperti XGBoost dan Gradient Boosting sangat mampu memanfaatkan banyak fitur, terutama fitur buatan (engineered) yang mengandung informasi penting.

Hubungan Scalling dan Feature Engineering

Pada model tree-based seperti XGBoost dan Gradient Boosting, proses scaling tidak wajib dilakukan karena pohon keputusan tidak sensitif terhadap skala fitur. Oleh karena itu, saat melakukan feature engineering, kita lebih bebas menggunakan nilai asli (aktual) tanpa perlu menyesuaikan skala. Hal ini memungkinkan model memanfaatkan perbedaan nilai numerik secara lebih alami dan akurat.



Fitur Yang di Tambahkan

Fitur	Penjelasan Lengkap
harga_setelah_diskon	Merupakan hasil dari harga asli produk dikalikan dengan persentase sisa setelah diskon. Fitur ini penting karena mewakili nilai akhir transaksi yang sebenarnya dibayar pelanggan, dan dapat berpengaruh terhadap kecepatan pemrosesan pengiriman.
berat_per_diskon	Rasio antara berat produk dengan jumlah diskon yang diberikan. Fitur ini menangkap efisiensi logistik—apakah diskon besar diberikan pada produk ringan atau berat, yang dapat memengaruhi strategi pengiriman dan keterlambatan.
produk_ringan	Fitur biner (0/1) yang mengkategorikan apakah produk tergolong ringan (berat < 1000 gram). Produk ringan kemungkinan lebih cepat diproses dan dikirim, sehingga berhubungan dengan prediksi pengiriman tepat waktu.
diskon_tinggi	Fitur biner yang menandai apakah sebuah produk mendapatkan diskon lebih dari 50%. Produk dengan diskon besar bisa menandakan promosi atau clearance sale yang mungkin memengaruhi prioritas pengiriman.
skor_pelanggan	Merupakan selisih antara rating yang diberikan pelanggan dan jumlah panggilan ke layanan pelanggan. Ini merefleksikan kepuasan relatif pelanggan; semakin positif nilainya, makin puas pelanggan terhadap layanan.
high_value_order	Fitur biner yang menunjukkan apakah transaksi bernilai tinggi sekaligus produk penting (harga > 10.000 dan kepentingan = "Tinggi"). Order semacam ini bisa memiliki prioritas logistik lebih tinggi atau pengaruh terhadap SLA (Service Level Agreement).
is_frequent_buyer	Fitur biner untuk menunjukkan pelanggan yang telah melakukan > 5 pembelian sebelumnya. Pelanggan yang loyal biasanya mendapat perlakuan khusus atau layanan lebih baik, sehingga bisa berdampak pada ketepatan waktu pengiriman.
produk_penting_dan_ringan	Kombinasi logis antara dua aspek—kepentingan tinggi dan bobot ringan. Produk yang penting namun ringan kemungkinan lebih cepat diproses dan memiliki urgensi tinggi dalam distribusi.
customer_loyalty	Skor gabungan dari pembelian sebelumnya dan rating pelanggan, yang dinormalisasi ke skala 0–1. Ini adalah representasi kuantitatif dari loyalitas pelanggan, dan bisa digunakan untuk mengidentifikasi apakah layanan memperhatikan pelanggan VIP dalam proses pengiriman.



TAMBAHAN

Outlier handling dilakukan juga pada fitur hasil rekayasa seperti berat_per_diskon, skor_pelanggan, dan juga fitur asli seperti diskon_ditawarkan menggunakan teknik capping berbasis IQR.

Tujuannya adalah untuk mengurangi dampak nilai ekstrem yang bisa mengganggu distribusi data, namun tetap menjaga data tetap utuh tanpa menghapus baris. Penanganan ini dilakukan setelah feature engineering, sehingga hasil transformasi tetap konsisten dan model dapat belajar dari pola yang lebih stabil.

Modelling



Data Splitting

```
from sklearn.model_selection
# Separate features and target
X = df_train.drop(columns=[
y = df_train['pengiriman_tep
# Split train data (80% train
X_train, X_val, y_train, y_
# Test set (features only)
X_test = df_test # No target
```

Membagi Data untuk Evaluasi Model

Memisahkan data menjadi data latih (train) dan data validasi (validation) agar model dapat dievaluasi secara objektif sebelum diuji pada data asli (test).

Alasan:

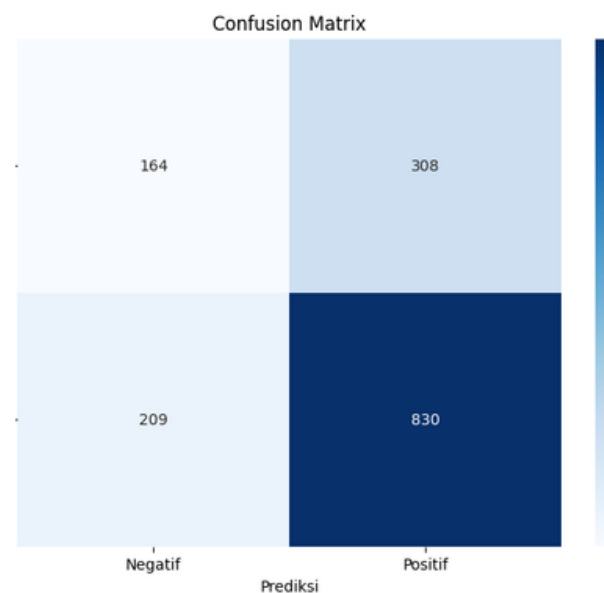
- Menghindari overfitting – model yang terlalu bagus di data latih tapi gagal di data baru.
- Memberikan gambaran performa model pada data yang belum pernah dilihat sebelumnya.

Tambahan

Data test (df_test) hanya berisi fitur → digunakan untuk prediksi akhir tanpa label target.



Random Forest



Precision sebesar 72.93% dan recall 79.88% menunjukkan bahwa model cukup baik dalam mendeteksi pengiriman tepat waktu, meskipun ada sedikit ketidakseimbangan antara ketepatan dan sensitivitas.

```
outlier handling untuk df_test
--- Outlier Handling pada df_test ---
outlier_cols:
l in ['mode_pengiriman', 'pembelian
Transformasi logaritmik (tambahkan
f_test[col] = np.log1p(df_test[col]
rint(f" - Kolom '{col}': Transform
col == 'diskon_ditawarkan':
rint(f" - Kolom '{col}': Capping")

ukan capping setelah transformasi (
st = cap_outliers_iqr(df_test, col)
```

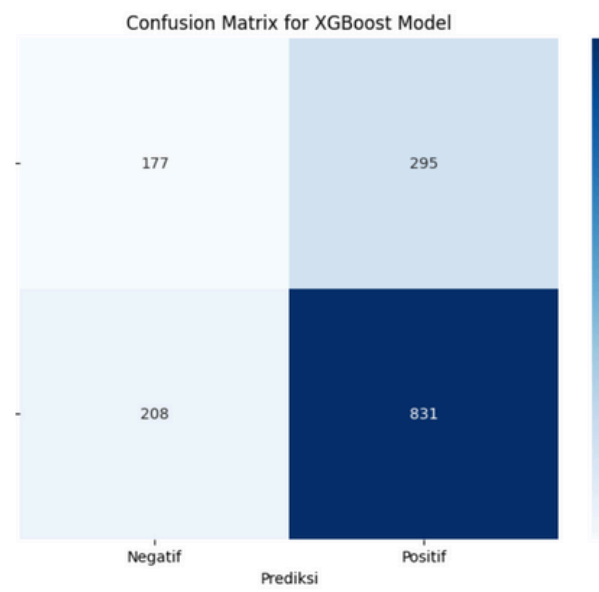
Model Random Forest menghasilkan akurasi 65.78%, yang menunjukkan performa moderat dalam klasifikasi pengiriman tepat waktu. Namun, akurasi saja tidak cukup untuk menilai kualitas model secara menyeluruh.

Random Forest Model Metr
Accuracy: 0.657842488418
Precision: 0.72934973637
Recall: 0.79884504331087
F1 Score: 0.762517225539
AUC Score: 0.71455196489

Dengan F1-score 76.25% dan AUC 71.45%, model menunjukkan performa yang layak namun masih bisa ditingkatkan, misalnya melalui tuning hyperparameter atau mencoba model lain seperti XGBoost.



XGBoost



```
seaborn as sns
matplotlib.pyplot as plt

# Initialize and train the XGBoost model
lcl = XGBClassifier(use_label_encoder=False, eval_metric='logloss')
lcl.fit(X_train, y_train)

# Predictions
y_preds_xgb = xgb_model.predict(X_val)
y_probs_xgb = xgb_model.predict_proba(X_val)[:, 1] # Get probabilities

# Calculate metrics
accuracy_xgb = accuracy_score(y_val, y_preds_xgb)
precision_xgb = precision_score(y_val, y_preds_xgb)
recall_xgb = recall_score(y_val, y_preds_xgb)
f1_xgb = f1_score(y_val, y_preds_xgb)
auc_xgb = roc_auc_score(y_val, y_probs_xgb)

# Print the results
print('XGBoost Model Metrics:')
print('Accuracy:', accuracy_xgb)
print('Precision:', precision_xgb)
print('Recall:', recall_xgb)
print('F1 Score:', f1_xgb)
print('AUC Score:', auc_xgb)
```

```
parameters: {'use_label_encoder': False}
```

warnings.warn(msg, UserWarning)

XGBoost Model Metrics:
Accuracy: 0.667107875579
Precision: 0.73801065719
Recall: 0.79980750721847
F1 Score: 0.767667436489
AUC Score: 0.72664801552

Con

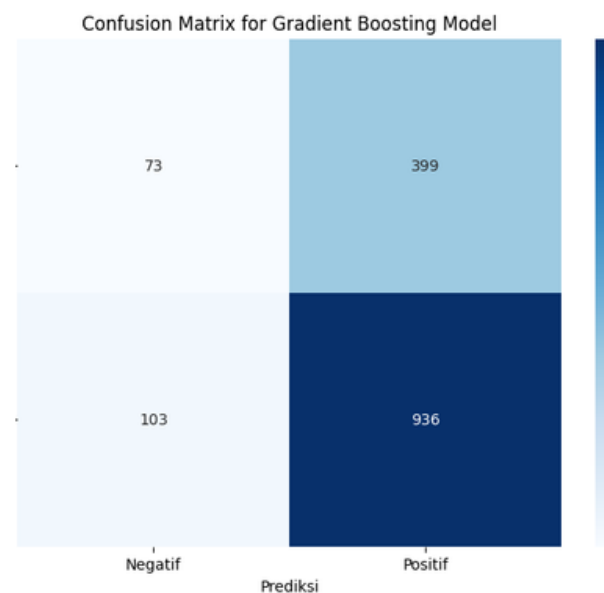
Nilai precision sebesar 73.80% dan recall sebesar 79.98% menunjukkan bahwa model cenderung sensitif dan cukup akurat dalam mengidentifikasi pengiriman yang tepat waktu. Kombinasi ini menghasilkan F1-score sebesar 76.76%, yang berarti keseimbangan antara ketepatan dan sensitivitas cukup baik.

Model XGBoost menghasilkan akurasi sebesar 66.71%, sedikit lebih tinggi dari Random Forest sebelumnya. Hal ini menunjukkan bahwa model mampu memprediksi label dengan benar pada sekitar dua pertiga data validasi. Meskipun belum optimal, peningkatan ini menunjukkan potensi XGBoost dalam menangani kompleksitas data lebih baik.

Dengan AUC score sebesar 72.66%, XGBoost memiliki kemampuan moderat dalam membedakan antara pengiriman tepat waktu dan terlambat. Secara keseluruhan, performa ini sudah baik dan bisa ditingkatkan lebih lanjut melalui hyperparameter tuning, feature selection, atau ensemble dengan model lain untuk mendorong skor lebih tinggi.



Gradient Boosting



```
seaborn as sns
matplotlib.pyplot as plt

# Initialize and train the XGBoost model
lcl = XGBClassifier(use_label_encoder=False, eval_metric='logloss')
lcl.fit(X_train, y_train)

# Predictions
y_preds_xgb = xgb_model.predict(X_val)
y_probs_xgb = xgb_model.predict_proba(X_val)[:, 1] # Get probabilities

# Evaluate metrics
acc_xgb = accuracy_score(y_val, y_preds_xgb)
prec_xgb = precision_score(y_val, y_preds_xgb)
rec_xgb = recall_score(y_val, y_preds_xgb)
f1_xgb = f1_score(y_val, y_preds_xgb)
auc_xgb = roc_auc_score(y_val, y_probs_xgb)

# Print the results
print('XGBoost Model Metrics:')
print('Accuracy:', acc_xgb)
print('Precision:', prec_xgb)
print('Recall:', rec_xgb)
print('F1 Score:', f1_xgb)
print('AUC Score:', auc_xgb)
```

```
plt.show()
```

Gradient Boosting Model Metrics:
Accuracy: 0.6677696889477
Precision: 0.701123595505
Recall: 0.900866217516843
F1 Score: 0.78854254422914
AUC Score: 0.727065015252

Confusion

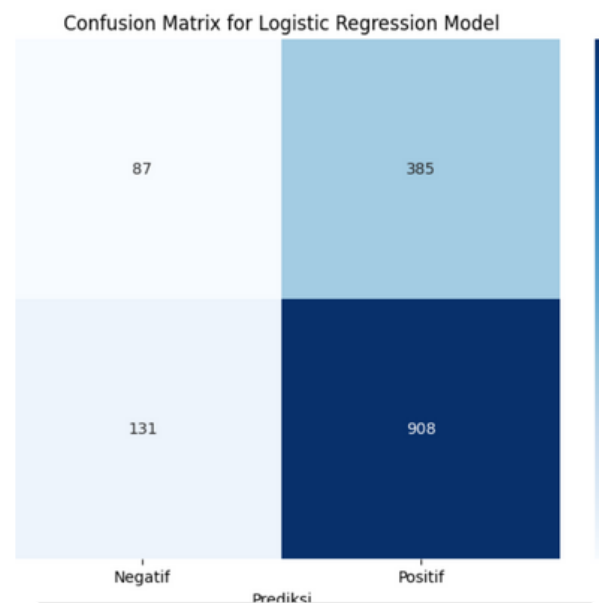
Dengan recall mencapai 90.09%, Gradient Boosting sangat efektif dalam mengenali pengiriman yang tepat waktu. Namun, presisi-nya sebesar 70.11% menunjukkan bahwa model masih menghasilkan false positive dalam jumlah yang cukup besar. Meski begitu, F1-score sebesar 78.85% merupakan salah satu yang tertinggi sejauh ini, menunjukkan keseimbangan antara ketepatan dan sensitivitas model.

Model Gradient Boosting menunjukkan performa yang solid dengan akurasi sebesar 66.78%, yang sebanding dengan model lain seperti XGBoost dan Random Forest. Meskipun akurasinya belum sangat tinggi, model ini konsisten dalam memprediksi dengan cukup baik pada data validasi.

Nilai AUC score sebesar 72.71% menunjukkan bahwa model cukup baik dalam membedakan antara pengiriman tepat waktu dan yang terlambat. Secara keseluruhan, Gradient Boosting cocok digunakan saat fokus utama adalah menangkap sebanyak mungkin pengiriman tepat waktu, meskipun ada trade-off berupa meningkatnya false positive. Model ini masih bisa ditingkatkan lagi dengan tuning hyperparameter atau feature selection yang lebih optimal.



Logistic Regression



```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, roc_auc_score
import seaborn as sns
import matplotlib.pyplot as plt

# Initialize and train the Logistic Regression model
log_reg_model = LogisticRegression(random_state=42)
log_reg_model.fit(X_train, y_train)

# Predictions
y_preds_log_reg = log_reg_model.predict(X_val)
y_probs_log_reg = log_reg_model.predict_proba(X_val)[:, 1]

# Calculate metrics
accuracy_log_reg = accuracy_score(y_val, y_preds_log_reg)
precision_log_reg = precision_score(y_val, y_preds_log_reg)
recall_log_reg = recall_score(y_val, y_preds_log_reg)
f1_log_reg = f1_score(y_val, y_preds_log_reg)
auc_log_reg = roc_auc_score(y_val, y_probs_log_reg)

# Print the results
print("Logistic Regression Model Metrics:")
```

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
n_iter_i = _check_...
Logistic Regression
Accuracy: 0.65850436
Precision: 0.7022428
Recall: 0.8739172281
F1 Score: 0.77873076
AUC Score: 0.6958124

Conf

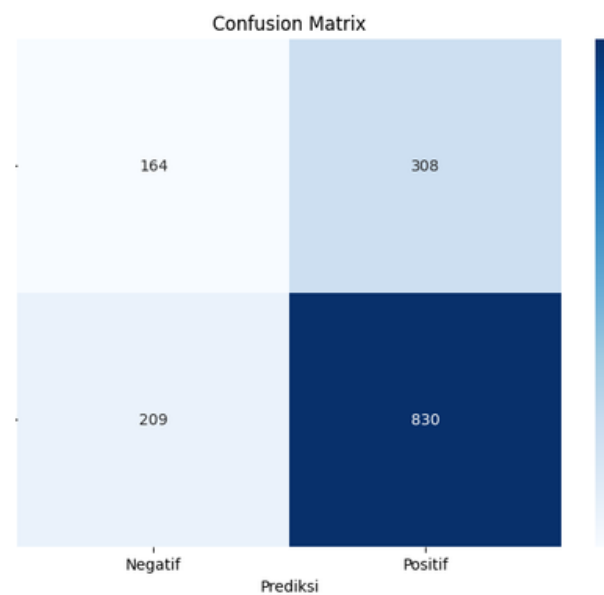
Model Logistic Regression menghasilkan akurasi sebesar 65.85%, sedikit di bawah Gradient Boosting dan XGBoost. Meskipun sederhana dan linier, model ini tetap memberikan performa yang cukup stabil dan layak sebagai baseline.

Nilai recall sebesar 87.39% cukup tinggi, menunjukkan bahwa model sangat sensitif dalam mengenali kasus pengiriman yang tepat waktu. Namun, precision sebesar 70.22% menunjukkan adanya cukup banyak prediksi "tepat waktu" yang ternyata salah. Meski begitu, F1-score sebesar 77.87% masih tergolong bagus dan cukup kompetitif dibanding model yang lebih kompleks.

Sayangnya, AUC score 69.58% adalah yang paling rendah di antara semua model yang kamu uji, menunjukkan bahwa kemampuan pemisahan antar kelas masih terbatas. Kesimpulannya, Logistic Regression cocok sebagai model baseline cepat dan sederhana, namun masih kalah dalam klasifikasi kompleks dibanding model tree-based seperti XGBoost dan Gradient Boosting.



Random Forest



Precision sebesar 72.93% dan recall 79.88% menunjukkan bahwa model cukup baik dalam mendeteksi pengiriman tepat waktu, meskipun ada sedikit ketidakseimbangan antara ketepatan dan sensitivitas.

```
outlier handling untuk df_test
--- Outlier Handling pada df_test ---
outlier_cols:
l in ['mode_pengiriman', 'pembelian
Transformasi logaritmik (tambahkan
f_test[col] = np.log1p(df_test[col]
print(f" - Kolom '{col}': Transform
col == 'diskon_ditawarkan':
print(f" - Kolom '{col}': Capping")

lakukan capping setelah transformasi (
st = cap_outliers_iqr(df_test, col)
```

Model Random Forest menghasilkan akurasi 65.78%, yang menunjukkan performa moderat dalam klasifikasi pengiriman tepat waktu. Namun, akurasi saja tidak cukup untuk menilai kualitas model secara menyeluruh.

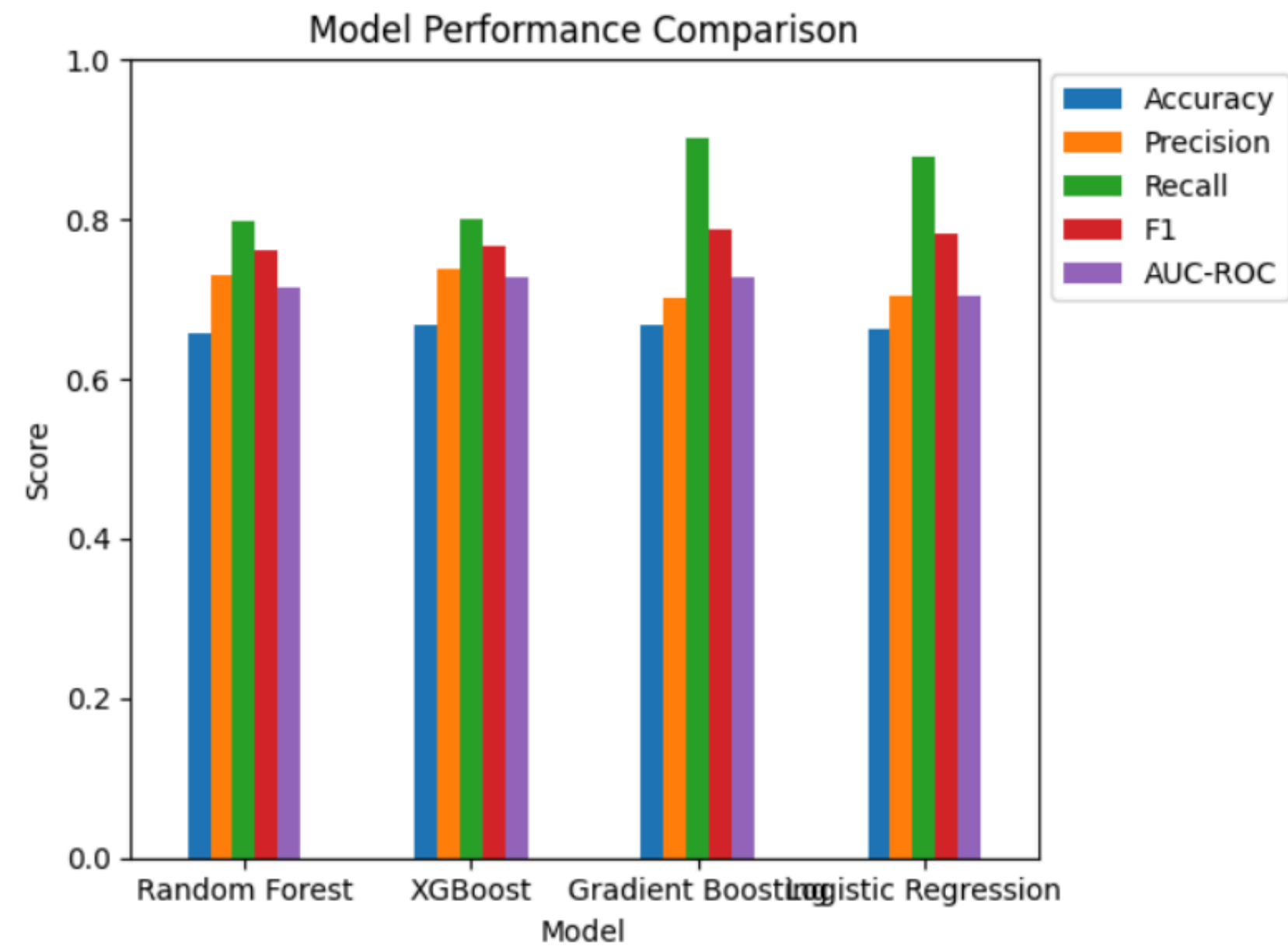
Random Forest Model Metrics
Accuracy: 0.657842488418
Precision: 0.72934973637
Recall: 0.79884504331087
F1 Score: 0.762517225539
AUC Score: 0.71455196489

Dengan F1-score 76.25% dan AUC 71.45%, model menunjukkan performa yang layak namun masih bisa ditingkatkan, misalnya melalui tuning hyperparameter atau mencoba model lain seperti XGBoost.

Evaluation



Visualisasi Perbandingan Model



Hasil Analisis Model

Setelah menguji empat model berbeda—Random Forest, XGBoost, Gradient Boosting, dan Logistic Regression—dapat disimpulkan bahwa Gradient Boosting memberikan performa terbaik secara keseluruhan. Meskipun akurasi tidak terlalu jauh berbeda dengan model lain (66.78%), model ini unggul signifikan dalam metrik yang lebih penting seperti recall dan F1 score.

Gradient Boosting berhasil mencapai recall tertinggi sebesar 90.09%, yang berarti model sangat efektif dalam menangkap seluruh kasus pengiriman tepat waktu. Hal ini sangat penting apabila konteks permasalahan lebih memprioritaskan deteksi pengiriman yang berhasil agar tidak salah diklasifikasikan sebagai terlambat. Selain itu, F1 Score sebesar 78.85% menunjukkan keseimbangan optimal antara precision dan recall, menjadikannya paling stabil dan andal dibanding model lainnya.

Meskipun XGBoost dan Logistic Regression memberikan precision yang sedikit lebih tinggi, keduanya tidak mampu menandingi F1 score dan recall dari Gradient Boosting. Ditambah lagi, Gradient Boosting juga mencatat AUC tertinggi (72.71%), menunjukkan kemampuannya yang lebih baik dalam membedakan dua kelas target. Dengan mempertimbangkan semua metrik evaluasi, Gradient Boosting menjadi pilihan model terbaik untuk digunakan dalam prediksi pengiriman tepat waktu pada dataset ini.



Score Submission



Final Submit Abil.csv

Complete · 18s ago

0.99877



sMAPE Public Score



Hasil hampir sempurna pada skor Kaggle menggunakan Gradient Boosting menyoroti keefektifan model ini dalam menangkap pola kompleks pada data. Dengan menggabungkan beberapa model sederhana secara iteratif, Gradient Boosting mengurangi kesalahan pelatihan dan prediksi, menghasilkan hasil yang lebih akurat dan andal.